16 Interfaces web

Vamos a desarrollar aplicaciones web escritas en Python, para lo que usaremos el módulo **flask**.

Para ello aprenderemos los diferentes tipos de páginas web y hablaremos sobre algunas de las tecnologías básicas para el mundo Web.

Comentaremos temas relacionados con seguridad y rendimiento.

Interfaces web y seguridad básica

Servidores Web

En este tema vamos a trabajar con todo lo relacionado con servidores web. Haremos algunos ejemplo de cómo montar una página web que va a publicar unos contenidos muy sencillos y posteriormente vamos a trabajar también con servicios web recuperando el contenido que otras páginas web publican.

Trabajaremos tanto la creación de sencillas aplicaciones web como el uso y acceso de servicios web.

Enfocaremos el contenido hacia la programación Python, dejando para un nivel superior el aspecto (CSS) como el uso de javascript, lo que nos obligaría a desviarnos de nuestra tarea principal que es Python.

Antes de adentrarnos en el uso de Flask, vamos a intentar dar una visión general sobre las tecnologías que se usan en programación web.

Introducción a las Tecnologías en Programación Web

La programación web es un campo dinámico y esencial en el desarrollo de aplicaciones modernas, que permite crear sitios y servicios interactivos accesibles a través de internet. En este capítulo, nos enfocaremos en la programación web con Python utilizando **Flask**, un framework ligero y flexible que facilita la construcción de aplicaciones web escalables.

Antes de sumergirnos en los detalles de Flask, es importante entender el panorama general de las tecnologías involucradas en el desarrollo web, distinguiendo entre dos componentes clave: el **frontend** y el **backend**. Esta división refleja cómo se estructuran las

aplicaciones web, donde el frontend se encarga de la interacción con el usuario y el backend maneja la lógica interna y los datos.

Diferencia entre Frontend y Backend

- **Frontend (Lado del Cliente):** Se refiere a la parte de la aplicación que el usuario ve e interactúa directamente en su navegador o dispositivo. Su objetivo principal es proporcionar una interfaz intuitiva, atractiva y responsive, que responda rápidamente a las acciones del usuario. El frontend se ejecuta en el cliente (el navegador web), lo que significa que no depende directamente del servidor para procesar cada interacción básica. En resumen, es lo que "se ve" y "se toca" en una página web.

- **Backend (Lado del Servidor)**: Es la parte invisible para el usuario, que se ejecuta en servidores remotos. Se encarga de la lógica de negocio, el procesamiento de datos, la autenticación, la gestión de bases de datos y la comunicación con el frontend. El backend responde a las solicitudes del frontend, procesa la información y devuelve resultados, asegurando que la aplicación funcione de manera segura y eficiente. A diferencia del frontend, el backend maneja operaciones complejas que requieren recursos del servidor, como consultas a bases de datos o integraciones con otros servicios.

Esta separación permite un desarrollo modular: equipos diferentes pueden trabajar en frontend y backend simultáneamente, y facilita la escalabilidad y el mantenimiento de la aplicación.

Tecnologías en el Frontend

El frontend se basa en tecnologías estándar que se interpretan directamente en el navegador. Las principales incluyen:

- **HTML (HyperText Markup Language)**: Es el esqueleto de cualquier página web. Define la estructura y el contenido, como encabezados, párrafos, listas e imágenes. Sin HTML, no habría una base para renderizar la información.

- **CSS (Cascading Style Sheets)**: Se encarga del diseño y la apariencia visual. Permite estilizar elementos, como colores, fuentes, layouts responsivos (adaptables a diferentes tamaños de pantalla) y animaciones. Frameworks como Bootstrap o

Tailwind CSS aceleran el desarrollo al proporcionar componentes predefinidos y estilos reutilizables.

- **JavaScript:** Es el lenguaje de programación del frontend, que añade interactividad y dinamismo. Permite manipular el DOM (Document Object Model), manejar eventos (como clics o envíos de formularios) y realizar peticiones asíncronas (AJAX) al backend sin recargar la página. Bibliotecas y frameworks populares incluyen:

- **React.js**: Desarrollado por Facebook, ideal para interfaces de usuario complejas y componentes reutilizables.

- **Vue.js o Angular**: Opciones alternativas para construir aplicaciones de una sola página (SPA) con estado reactivo.

- **jQuery**: Una biblioteca más antigua pero útil para simplificar manipulaciones DOM en proyectos simples.

Estas tecnologías se combinan para crear experiencias de usuario fluidas, y herramientas como Webpack o Vite ayudan en la compilación y optimización del código frontend.

Tecnologías en el Backend

El backend involucra lenguajes de programación, servidores y bases de datos para manejar la lógica del servidor. En el contexto de este capítulo, nos centraremos en Python con Flask, pero es útil conocer el ecosistema general:

- **Lenguajes de Programación**: Python es uno de los más populares por su simplicidad y legibilidad. Otros comunes incluyen Node.js (basado en JavaScript), Java (con Spring), Ruby (con Rails) o PHP (con Laravel). Python destaca en backend por su vasto ecosistema de bibliotecas para datos, IA y web.

- **Frameworks Web**: Facilitan el desarrollo al proporcionar estructuras predefinidas. En Python:

  - **Flask**: Un microframework minimalista, ideal para aplicaciones pequeñas a medianas. Ofrece flexibilidad para construir APIs RESTful, manejar rutas y plantillas, sin imponer estructuras rígidas. Es perfecto para principiantes y proyectos que requieren personalización.
  - **Django**: Un framework más completo y "batteries-included", con ORM integrado, autenticación y admin panel, pero menos ligero que Flask.

- **Servidores y Despliegue**: Herramientas como Gunicorn o uWSGI sirven aplicaciones Python en producción. Plataformas de despliegue incluyen Heroku, AWS, Google Cloud o Vercel para hosting escalable.

- **Bases de Datos**: Almacenan y gestionan datos persistentes. Opciones incluyen:

  - **SQL**: Como PostgreSQL, MySQL o SQLite (ideal para desarrollo local con Flask), para datos estructurados con relaciones.

  - **NoSQL**: Como MongoDB, para datos flexibles y escalables.

  - **Otras Tecnologías**: APIs para integración (REST o GraphQL), autenticación (JWT, OAuth), y herramientas como Redis para caching o Celery para tareas en background.

En resumen, mientras el frontend se enfoca en la presentación y la usabilidad con HTML, CSS y JavaScript, el backend asegura la funcionalidad y seguridad con lenguajes como Python y frameworks como Flask. Esta sinergia es lo que hace posible aplicaciones web robustas. En las secciones siguientes, explicaremos cómo implementar Flask para crear tu primera aplicación web, integrando estos conceptos en la práctica.

Introducción a las Webs Estáticas y Dinámicas

En el desarrollo web, las páginas se clasifican principalmente en dos categorías: **estáticas** y **dinámicas**. Una web estática consiste en archivos HTML, CSS y JavaScript pregenerados que no cambian en función del usuario o de datos en tiempo real. Por el contrario, una web dinámica genera contenido de manera personalizada en el servidor, respondiendo a solicitudes del usuario, bases de datos o interacciones en tiempo real. Un ejemplo común de web dinámica es una aplicación desarrollada en Python con Flask, un framework ligero que permite crear servidores web que procesan lógica en el backend.

A continuación, explico detalladamente las diferencias en el desarrollo y la implantación (despliegue), los tipos de páginas para las que son adecuadas, y las ventajas e inconvenientes de cada una. Usaré una estructura clara para facilitar la comparación.

16 Interfaces web

Diferencias en el Desarrollo

El desarrollo es el proceso de creación del código y la lógica de la web. Aquí radican las principales distinciones:

Web Estática:

- Se basa en archivos fijos: HTML para la estructura, CSS para el estilo y JavaScript para interacciones del lado del cliente (frontend).
- No requiere un lenguaje de programación en el servidor. Puedes usar generadores estáticos como Hugo, Jekyll o Eleventy para automatizar la creación de páginas a partir de plantillas.
- El contenido se define de antemano y no cambia dinámicamente. Cualquier actualización implica editar archivos manualmente o regenerar el sitio.
- Ejemplo: Un sitio web simple con páginas como "Inicio", "Acerca de" y "Contacto", donde el contenido es el mismo para todos los usuarios.

Web Dinámica (ej. Python con Flask):

- Involucra un backend que procesa solicitudes HTTP. Flask es un microframework que permite definir rutas (endpoints) en Python, como `@app.route('/')`, para manejar peticiones GET/POST.
- Requiere integración con bases de datos (ej. SQLAlchemy para SQLite o PostgreSQL), autenticación de usuarios, procesamiento de formularios y lógica condicional.
- El contenido se genera en tiempo real: por ejemplo, un usuario inicia sesión y ve datos personalizados extraídos de una BD.
- Desarrollo más complejo: Incluye manejo de errores, seguridad (ej. contra inyecciones SQL), sesiones y APIs RESTful.

En resumen, el desarrollo de una web estática es más rápido y simple para principiantes, mientras que una dinámica requiere conocimientos en programación server-side y manejo de estados.

16 Interfaces web

Diferencias en la Implantación (Despliegue)

La implantación se refiere a cómo se sube y se mantiene la web en un servidor accesible al público.

Web Estática:

- Fácil y económica: Solo subes archivos a un hosting estático como GitHub Pages, Netlify, Vercel o Amazon S3. No necesitas un servidor dedicado; se sirven directamente desde un CDN (Content Delivery Network).
- No hay procesos en ejecución continua. El despliegue es un simple "push" de archivos, y las actualizaciones se propagan rápidamente.
- Escalabilidad automática: Soporta alto tráfico sin costos adicionales, ya que no hay cómputo en el servidor.

Web Dinámica (ej. Python con Flask):

- Requiere un servidor que ejecute código: Plataformas como Heroku, AWS EC2, DigitalOcean o Render. Debes configurar un entorno con Python, instalar dependencias (via pip) y usar un servidor WSGI como Gunicorn.
- Involucra configuración de bases de datos, variables de entorno (ej. para claves API) y monitoreo de logs.
- Despliegue más complejo: Usa CI/CD (ej. GitHub Actions) para automatizar, pero puede requerir downtime durante actualizaciones. Para producción, se recomienda usar contenedores como Docker para portabilidad.

La implantación estática es "set it and forget it", ideal para bajo mantenimiento, mientras que la dinámica demanda monitoreo constante y recursos computacionales.

16 Interfaces web

Tipos de Páginas Adecuadas para Cada Una

Web Estática:

- Adecuada para sitios informativos o de presentación donde el contenido no cambia frecuentemente y no requiere interacción personalizada.Ejemplos: Portafolios personales, blogs estáticos, landing pages de marketing, sitios de documentación (ej. manuales de productos), páginas corporativas básicas o sitios de eventos pasados.

- Ideal para proyectos con bajo presupuesto, alto rendimiento y enfoque en SEO, ya que las páginas se indexan fácilmente.

Web Dinámica (ej. Python con Flask):

- Adecuada para aplicaciones interactivas que manejan datos en tiempo real, usuarios registrados o contenido generado por el usuario.Ejemplos: Redes sociales, e-commerce (ej. carrito de compras dinámico), dashboards administrativos, foros, aplicaciones de gestión (ej. CRM), sitios con búsqueda avanzada o integración con APIs externas (ej. pagos con Stripe).
- Perfecta para escenarios donde la personalización es clave, como recomendaciones basadas en datos o actualizaciones en vivo.
- En general, elige estática si el sitio es "leer solo", y dinámica si implica "leer y escribir" datos.

Ventajas e Inconvenientes

Para una comparación clara, uso una tabla:

16 Interfaces web

Aspecto	Web Estática	Web Dinámica (ej. Flask)
Ventajas	➢ **Alta velocidad**: Carga rápida ya que no hay procesamiento en servidor. ➢ **Seguridad**: Menos vulnerabilidades (no hay BD ni código server-side expuesto). ➢ **Bajo costo**: Hosting gratuito o barato, sin consumo de CPU. ➢ **Fácil escalabilidad**: Maneja picos de tráfico sin problemas. ➢ **Buen SEO**: Páginas pre-renderizadas.	➢ **Flexibilidad**: Contenido personalizado y dinámico. ➢ **Interactividad**: Manejo de usuarios, formularios y datos en tiempo real. ➢ **Integración**: Fácil conexión con BD, APIs y servicios externos. ➢ **Actualizaciones fáciles**: Cambios en datos sin regenerar todo el sitio. ➢ **Potencial para apps complejas**: Escalable con microservicios.
Inconvenientes	➢ **Limitada interactividad:** No soporta login, comentarios o actualizaciones en vivo sin herramientas externas (ej. Disqus para comentarios). ➢ **Actualizaciones manuales**: Cualquier cambio requiere redeploy. ➢ **No para datos dinámicos**: Inadecuada para e-commerce o redes sociales. ➢ **Dependencia de generadores**: Para sitios grandes, puede ser tedioso sin un CMS estático.	➢ **Mayor complejidad**: Requiere más código y depuración. ➢ **Costos más altos**: Servidores, BD y mantenimiento. ➢ **Rendimiento**: Puede ser más lento si no se optimiza (ej. caching con Redis). ➢ **Seguridad**: Riesgos como inyecciones o fugas de datos si no se maneja bien. ➢ **Escalabilidad**: Necesita balanceo de carga para alto tráfico.

Conclusión

En resumen, las webs estáticas son ideales para simplicidad, velocidad y bajo mantenimiento, perfectas para sitios informativos. Las dinámicas, como las desarrolladas con Flask, ofrecen poder y personalización, pero a costa de mayor complejidad y recursos. La elección depende del propósito: si tu proyecto necesita interacción y datos variables, opta por dinámica; de lo contrario, estática es más eficiente.

Servidor Web con Flask

Vamos a ver ahora un ejemplo sencillo de servidor web que publica un contenido.

Tenemos diferentes alternativas para hacerlo:

- Usando sockets y threads donde trabajamos al nivel más bajo.
- Usando las clases http.server, trabajaríamos a un nivel intermedio
- Django: se trata de un framework profesional y muy avanzado pensado para crear sitios web complejos.
- **Flask**: un framework sencillo y básico con el que podemos hacer muchas cosas de una manera simple.
- FastApi: un framework moderno de alto rendimiento para construir APIs o páginas sencillas.

Vamos a usar **Flask**, un framework de Python, sencillo y básico pensado para crear webs dinámicas de una forma sencilla. Decimos que es básico porque si bien nos permite crear páginas dinámicas, servicios webs, trabajar con templates,... no está diseñado para trabajar en modelos más complejos como el MVC (Master View Controller) ni integrarse fácilmente con el modelo relacional de bases de datos.

Flask, sí que incluye un servidor web propio y permite el uso y la depuración de todo el proceso.

Primer ejemplo: "Hello Flask"

Vamos a crear una sencilla aplicación web: la típica Hello World, que llamaremos "Hello Flask" ;)

16 Interfaces web

Empezamos instalando Flask. Si trabajamos desde el terminal ejecutaremos:

```
pip3 install flask
```

Para trabajar aplicaciones web, crearemos una carpeta de aplicación "webApp" donde pondremos todos los ficheros necesarios:

```
mkdir webApp
```

y dentro creamos una versión de prueba de la aplicación en el fichero **app.py**

```
from flask import Flask, render_template

app = Flask("Primera prueba Flask")

@app.route('/')
def home():
   return render_template('home.html')

@app.route('/quien-soy')
def quien():
   return render_template('quien.html')

if __name__ == '__main__':
   app.run(host='0.0.0.0', debug=True)
```

creamos el directorio **templates** con los archivos:

home.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Title</title>
</head>
<body>
    <h1>¡Hola Mundo!</h1>
```

```
</body>
</html>
```

quien.html

```
<!DOCTYPE html>
<html>
<head>
   <title>Quien soy</title>
</head>
<body>
   <h1>Hola me llamo Antonio</h1>
   <a href="/">Al hacer clic aquí te lleva al home</a>
</body>
</html>
```

Podemos arrancar la aplicación web desde la consola con:

```
python3 app.py
```

Y nos avisa de que está funcionando

```
 * Serving Flask app 'Primera prueba Flask'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a
production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://192.168.98.168:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 107-958-678
```

16 Interfaces web

Ejemplo de uso de datos del usuario: Suma

Vamos a hacer ahora una sencilla web en la que tomaremos los datos que el usuario ponga en un formulario y mostraremos el resultado de la suma. Generamos el HTML desde el mismo código Python.

Este es el fichero del código que podemos llamar app.py

```
# Pagina web con flask
from flask import Flask, request

app = Flask('Hello Flask')

@app.route('/')
def index():
    return 'Hello Flask'

@app.route('/test')
def test():
    return 'test Flask'


@app.route('/suma',methods = ["GET", "POST"])
@app.route('/suma/',methods = ["GET", "POST"])
def suma():
    htmlCode = ""
    if request.method == "POST":
        print("POST") # Obtenemos datos y calcula
        sumando1 = int(request.form.get("Sumando1"))
        sumando2 = int(request.form.get("Sumando2"))
        htmlCode = "Resultado: " +  str(sumando1 +sumando2)
    else:
        print("GET") # Mostramos HTML
        htmlCode = '''<form action="/suma" method="POST">
                <label>Sumando 1:</label>
                <input type="text" name="Sumando1"/>
                <label>Sumando 2:</label>
                <input type="text" name="Sumando2"/><br/><br/>
                <input type="submit"/>
                </form>'''
    return htmlCode

if __name__ == '__main__':
    app.run(debug = False, host='127.0.0.1') # solo acceso local y puerto 5000
```

16 Interfaces web

Ejemplo: cálculo del IRPF

Tendremos un formulario donde el usuario pondrá el importe bruto de su sueldo y obtendrá todos los cálculos.

En este caso vamos a usar unas plantillas HTML (templates) que tenemos que incluir en la carpeta "**./templates**" de la carpeta de la aplicación. Usaremos una página **index.html** para la petición de datos y otra **index_resultado.html** para devolver los resultados.

Veamos la plantilla **index.html** que usaremos inicialmente para pedir los datos (método GET).

Dentro de una plantilla se puede incluir además del código HTML:

- **Variables** que estarán rodeadas por {{ variable }}. Aquí podemos usar variables, listas, diccionarios,....
- **Bucles for o sentencias if**, se indican con {% ... %} y que siguen una sintaxis similar a la de python
- **Comentarios**: todo el contenido que incluyamos entre estos símbolos {# ... #} se ignorará

```
<html lang="es">
<head><title>Cálculo de % de IRPF a pagar</title></head>
<body>
    <h2>{{ resultado_msg }}</h2>
    <form action="/" method="POST">
        <label>Sueldo Bruto</label>
        <input type="text" name="sueldo"/>
        <br/><br/>
        <input type="submit"/>
        </form>
</body>
</html>
```

Para procesar una plantilla llamaremos al método render, pasándole los valores de todas aquellas variables que han sido declaradas dentro de la plantilla. Por ejemplo

```
render_template('plantilla3.html',variable1=valor1,variable2=[v2,v3,v3])
```

Veamos el uso de una plantilla más compleja para mostrar el resultado **index_resultado.html**, donde usamos un bucle para mostrar

las diferentes filas del resultado que pasaremos al método render en una lista

```
<html lang="es">
<head><title>Cálculo de % de IRPF a pagar</title></head>
<body>
    <table>
        <tr><td><strong>Importe tramo</strong></td><td><strong>%
tramo</strong></td><td><strong>Importe pagado</strong></td></tr>
        {% for filaDato in filasDatos %}
        <tr><td>{{ filaDato[0] }}</td><td>{{ filaDato[1] }}</td><td>{{ filaDato[2] }}</td></tr>
        {% endfor %}
        <tr><td></td></tr>
        <tr><td><strong>Total</strong></td></tr>
        <tr><td>Sueldo Bruto</td><td>% efectivo</td><td>Impuestos Pagados</td></tr>
        <tr><td>{{ sueldo }}</td><td>{{ porcentaje_efectivo }}</td><td>{{ importe_pagado
}}</td></tr>
    </table>
<br/><br/>

<h2>{{ resultado_msg }}</h2>

<form action="/" method="POST">
        <label>Sueldo Bruto</label>
        <input type="text" name="sueldo"/>
        <br/><br/>
        <input type="submit"/>
        </form>

    </body>
</html>
```

El código de **irpf_app.py** queda así:

```
from flask import Flask, render_template, request, Response

tramos = ((12450,  0.19),
          (20200,  0.24),
          (35200,  0.30),
          (60000,  0.37),
          (300000, 0.45),
          (10e20,  0.47)) # Ponemos un máximo ficticio

def calculoIRPF(sueldo):
    impuesto_a_pagar = 0 # Iremos sumando el pago de cada tramo
    ya_pagado = 0  # guardamos lo ya pagado hasta el tramo anterior
    filasDatos = []
    for max_tramo, tipo_tramo in tramos: # Iteramos en los tramos
        tope_tramo = min(sueldo, max_tramo)  # Sera lo maximo por lo que
paguemos en el tramo
        importe_en_tramo = tope_tramo - ya_pagado  # Pagamos por esta cantidad
en este tramo
        pago_tramo = importe_en_tramo * tipo_tramo # Pago en este tramo
        impuesto_a_pagar +=  pago_tramo      # Acumulamos en el pago total
```

```
        filasDatos.append((importe_en_tramo, 100 * tipo_tramo, pago_tramo))
        ya_pagado = tope_tramo # actualizamos lo pagado
        if ya_pagado == sueldo:
            break # es nuestro ultimo tramo
    return filasDatos, ya_pagado, impuesto_a_pagar*100/ya_pagado,
impuesto_a_pagar


app = Flask('Hello Flask')

@app.route('/',methods = ["GET", "POST"])
def index():
    resultado_msg = "Introduzca su sueldo bruto"
    if request.method == "POST":
        sueldo = int(request.form.get("sueldo"))
        fSueldo = float(sueldo)
        filasDatos, ya_pagado, porcentaje_efectivo, importe_pagado =
calculoIRPF(fSueldo)

        return render_template('index_resultado.html',
            resultado_msg = resultado_msg,
            filasDatos = filasDatos,
            sueldo = ya_pagado,
            porcentaje_efectivo = porcentaje_efectivo,
            importe_pagado = importe_pagado)
    else:
        return render_template('index.html', resultado_msg = resultado_msg)


if __name__ == '__main__':
    app.run(debug = True, host='127.0.0.1') # solo acceso local y puerto 5000
```

El código de la web queda así

16 Interfaces web

Servicios Web

Un servicio web es una forma de acceder a un programa que se está ejecutando en otro ordenador remotamente mediante protocolo HTTP
Hay muchas formas de consumir un servicio, como por ejemplo lo que se conoce como **API REST**: una forma sencilla en que sólo tenemos que hacer una petición GET al servidor y en la URL que pedimos está toda la información.

Casi todos los servicios devuelven la información en un formato llamado **json** (muy similar a los diccionarios que hemos visto)
La forma en la construimos la URL y cómo nos devuelven los datos es lo que se conoce como API

Es normal que estos servicios nos requieran que nos demos de alta para usarlos. Cuando lo hacemos generalmente nos proporcionan un **Token o API Key,** que tenemos que incluir en las peticiones que hacemos. De esta manera pueden controlar el uso que hacemos de su servicio.

Búsqueda de emojis

Vamos a hacer un ejemplo con este servicio de búsqueda de emoji siguiendo los ejemplos

Si hacemos una búsqueda de 'computer' con esta url

https://emoji-api.com/emojis?search=computer&access_key=YOUR_API_KEY
obtendremos este resultado:


```
[{"slug":"desktop-computer","character":"\ud83d\udda5\ufe0f","unicodeName":"desk
top computer","codePoint":"1F5A5
FE0F","group":"objects","subGroup":"computer"},{"slug":"computer-mouse","charact
er":"\ud83d\uddb1\ufe0f","unicodeName":"computer mouse","codePoint":"1F5B1
FE0F","group":"objects","subGroup":"computer"},{"slug":"computer-disk","characte
r":"\ud83d\udcbd","unicodeName":"computer
disk","codePoint":"1F4BD","group":"objects","subGroup":"computer"}]
```


```
import request
r = requests.get('https://emoji-api.com/emojis?search=computer&access_key=YOUR_API_KEY')
datos_json=r.json()
for code in datos_json:
    print(code['character'] + '     ' + code['slug'])
```

16 Interfaces web

Vamos a hacer un ejemplo un poco más complicado donde vamos a filtrar por el parámetro subGroup:

Código

```
import request
r = requests.get('https://emoji-api.com/emojis?search=cloud&access_key=YOUR_API_KEY')
datos_json=r.json()
for code in datos_json:
    if code['subGroup'] == 'sky-weather':
        print(code['character'] + '     ' + code['slug'])
```

Buscadores

Veamos este sencillo ejemplo de uso del buscador duckduckgo

Código

```
import requests
r = requests.get("http://duckduckgo.com/?q=micropython&format=json").json()
print(r)
print(r['AbstractText'])
```

Pronóstico del tiempo

Veamos un ejemplo de cómo obtener un pronóstico del tiempo. Usaremos el servicio de http://openweathermap.org Para usarlo nos damos de alta como usuario, obteniendo una licencia gratuita que nos permite hacer 60 peticiones por hora o 1000000 al mes dándonos información sobre pronóstico horario, por días, etc... o sobre datos históricos

Código

```
import requests
r =
requests.get("http://api.openweathermap.org/data/2.5/weather?q=Gra
nada&appid=YOUR_API_KEY").json()
description = r["weather"][0]["description"]
temp = r["main"]["temp"] - 273.15
mainWeather = r['weather'][0]['main']
```

16 Interfaces web

Referencias:

[Pronóstico el tiempo by Intef]
[API-Keys]
[Ejemplo de consulta]

No es seguro api.openweathermap.org/data/2.5/forecast?id=2510911&appid=611bd899879bb31e6...

Dar formato al texto


```
{
  "cod": "200",
  "message": 0,
  "cnt": 40,
  "list": [
    {
      "dt": 1762441200,
      "main": {
        "temp": 292.67,
        "feels_like": 292.06,
        "temp_min": 292.58,
        "temp_max": 292.67,
        "pressure": 1018,
        "sea_level": 1018,
        "grnd_level": 1014,
        "humidity": 53,
        "temp_kf": 0.09
      },
      "weather": [
        {
          "id": 802,
          "main": "Clouds",
          "description": "scattered clouds",
          "icon": "03d"
        }
      ],
      "clouds": {
        "all": 47
      },
```


[Código de errores]